SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 3 – Vulnerability Detection

Course Code:	CSA5803	**Course Title:**	DEVSECOPS ENGINEERING AND APPLICATION SECURITY
CO Assessed:	CO3 – Precision (BL1, BL2, BL3)	**Assessment:**	Assessment Tool 3
Weightage:	40% of CIA	**Total Marks:**	100
CO 3 Statement:	Vulnerability Detection, Severity Analysis, Security Verification Workflow Simulation and Performance Evaluation		
Student Name:	M.GOKUL	**Reg. No.:**	192565067
Section / Batch:	2025	**Date:**	13/08/2026

Instructions to Students

- Perform vulnerability detection and classify the identified vulnerabilities based on their severity levels.
- Conduct severity analysis using appropriate metrics such as CVSS and document the impact of each vulnerability.
- Design and simulate a Security Verification Workflow covering detection, remediation, re-testing, and verification.
- Evaluate security and system performance before and after mitigation, and include relevant results, screenshots, and observations.

Question Type	Focus Area	Questions	Marks
Vulnerability Detection	Conceptual Understanding	Q1	Q1 –100
GRAND TOTAL:		**100 Marks**	

Code of Conduct

*I **M.GOKUL 192565067** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the Student: 

ASSESSMENT -9

Title

CI/CD Pipeline Automation with Integrated SAST, DAST, and SCA

2. Introduction

CI/CD automation helps developers build, test, and deploy applications automatically. Integrating SAST, DAST, and SCA into the pipeline allows security vulnerabilities to be detected before the application reaches production.

3. Problem Statement

Traditional CI/CD pipelines mainly focus on functionality and deployment. Security issues may remain undetected until later stages. Therefore, security testing must be integrated directly into the CI/CD pipeline.

4. Objectives

- Automate application security testing.
- Detect vulnerabilities early.
- Analyze vulnerability severity.
- Reduce security risks.
- Automate security verification before deployment.

5. Scope

The project covers source-code analysis, dynamic application testing, dependency analysis, vulnerability classification, remediation, re-testing, and automated security verification.

6. CI/CD Pipeline

A CI/CD pipeline automates the process of code → build → test → security scan → deployment. Security tools are added to the pipeline so that insecure code can be identified automatically.

7. DevSecOps

DevSecOps integrates security into the DevOps lifecycle. Instead of testing security only at the end, security checks are performed continuously during development and deployment.

8. SAST

Static Application Security Testing (SAST) examines application source code without executing the application. It can identify coding weaknesses such as insecure input handling and hard-coded secrets.

9. DAST

Dynamic Application Security Testing (DAST) tests a running application from the outside. It helps identify vulnerabilities that appear during application execution.

10. SCA

Software Composition Analysis (SCA) checks third-party libraries and dependencies used by an application. It identifies known vulnerabilities in outdated or insecure components.

11. SAST, DAST and SCA Integration

The three techniques provide different security views:

Technique	Testing Method	Main Purpose
SAST	Source code	Find coding vulnerabilities
DAST	Running application	Find runtime/web vulnerabilities
SCA	Dependencies	Find vulnerable libraries

12. Pipeline Workflow

The proposed workflow is:

Developer Commit → Build → SAST → SCA → Application Deployment → DAST → Severity Analysis → Security Gate → Deployment/Remediation

13. Vulnerability Detection

The pipeline automatically scans the application and identifies security vulnerabilities. Each detected vulnerability is recorded with its type, location, severity, and recommended remediation.

14. Vulnerability Classification

Detected vulnerabilities can be classified according to their severity:

- Critical
- High
- Medium
- Low

Critical and high-severity vulnerabilities should receive immediate attention.

15. CVSS

Common Vulnerability Scoring System (CVSS) provides a standardized method for measuring vulnerability severity. The score helps prioritize which vulnerabilities should be fixed first. The PDF specifically requires severity analysis using appropriate metrics such as CVSS.

16. Security Gate

A security gate is a pipeline condition that decides whether the build can continue. For example, the pipeline can fail when a critical or high-severity vulnerability is detected.

17. Vulnerability Remediation

After vulnerabilities are detected, developers fix the affected source code or dependencies. Examples include validating input, removing insecure code, and updating vulnerable libraries.

18. Re-testing

After remediation, the application is scanned again. SAST, DAST, or SCA is executed again to confirm that the previously detected vulnerability has been resolved.

19. Security Verification

Security verification confirms that the implemented fixes are effective and that the application satisfies the defined security requirements.

20. Before-Mitigation Analysis

Before mitigation, the system may contain multiple vulnerabilities. The initial scan results are recorded and used as the baseline for comparison.

21. After-Mitigation Analysis

After remediation and re-testing, the number and severity of vulnerabilities should decrease. The results are compared with the original baseline.

22. Performance Evaluation

The pipeline can be evaluated using:

- Number of vulnerabilities detected
- Number of vulnerabilities fixed
- Scan execution time
- Build execution time
- Pipeline success/failure
- Security improvement

The PDF specifically asks for evaluation of security and system performance before and after mitigation.

23. Expected Results

The expected result is an automated CI/CD pipeline that detects vulnerabilities using SAST, DAST, and SCA and prevents insecure builds from progressing when defined security conditions are not satisfied.

24. Advantages

- Early vulnerability detection
- Automated security testing
- Reduced manual effort
- Continuous security monitoring
- Faster remediation
- Improved application security
- Better integration of security into development

25. Conclusion

The proposed CI/CD Pipeline Automation with Integrated SAST, DAST, and SCA provides a continuous security verification mechanism. By detecting vulnerabilities, analyzing their severity, applying remediation, and performing re-testing, the system improves application security throughout the CI/CD lifecycle. This directly addresses the PDF's required workflow of detection → remediation → re-testing → verification → performance evaluation.

CI/CD Pipeline

A typical pipeline contains:

Code Commit → Build → SAST → SCA → Deploy to Test Environment → DAST → Security Gate → Verification → Deployment

Additional points:

- Pipeline execution is automated.
- Each stage produces results.
- Failed security checks can stop the pipeline.
- Security results can be stored for analysis.
- The process reduces manual intervention.

DevSecOps

- DevSecOps combines development, operations, and security.
- Security is introduced early in the software lifecycle.
- Developers receive security feedback earlier.
- Security testing becomes continuous.
- Vulnerabilities can be fixed before deployment.
- It supports the security verification workflow required by the assessment.

SAST

Static Application Security Testing

- Examines source code.
- Does not require the application to be running.
- Detects insecure coding practices.
- Identifies possible security weaknesses.
- Can be executed automatically in the CI pipeline.
- Results can identify the affected file and code location.
- Developers can fix issues before deployment.

DAST

Dynamic Application Security Testing

- Tests a running application.
- Simulates external security testing.
- Identifies runtime vulnerabilities.
- Checks application behavior.
- Can identify web-application security issues.
- Runs after the application is deployed to a test environment.

- Results can be used for remediation and re-testing.

SCA

Software Composition Analysis

- Examines third-party dependencies.
- Identifies known vulnerabilities in libraries.
- Detects outdated components.
- Helps developers identify risky dependencies.
- Supports dependency management.
- Can be executed automatically during CI.
- Vulnerable dependencies can be updated or replaced.

Integration of SAST, DAST and SCA

Method	Input	Main Purpose	Pipeline Stage
SAST	Source code	Detect coding vulnerabilities	Before deployment
SCA	Dependencies	Detect vulnerable components	Build/CI stage
DAST	Running application	Detect runtime vulnerabilities	Test environment

Together they provide broader vulnerability coverage.

12. Pipeline Workflow

Step 1

Developer commits code.

Step 2

CI system starts the pipeline.

Step 3

Application is built.

Step 4

SAST scans the source code.

Step 5

SCA checks dependencies.

Step 6

Application is deployed to a test environment.

Step 7

DAST scans the running application.

Step 8

All vulnerabilities are collected.

Step 9

Severity is analyzed.

Step 10

Security gate decides whether the pipeline continues.

Step 11

Developers remediate vulnerabilities.

Step 12

The pipeline performs re-testing.

Step 13

Security is verified.

Step 14

Final results are evaluated.

13. Vulnerability Detection

- Scan the source code.
- Scan application dependencies.
- Scan the running application.
- Record detected vulnerabilities.
- Identify vulnerability type.
- Identify affected component.
- Record severity.
- Record recommended remediation.
- Compare results before and after mitigation.

The PDF explicitly requires vulnerability detection and classification based on severity.